MCP PRODUCTION · SPEC REVISION 2026-07-28

Diagnose an ambiguous timeout

The flow sheet from lesson 2.1: what a timeout, a handler log, a receipt or a matching note each establish, and which identifier answers which question.

The example is an append to a shared notebook. A lost answer can hide a committed note. Request arrival, database commit, and response receipt are different observations; none should be inferred merely from the preceding one.

```
sequenceDiagram
    participant C as Client
    participant S as Server
    participant D as Database
    C-»S: Append note with operation key
    S-»D: Commit note and operation receipt
    D--»S: Commit completed
    Note over S,C: Response does not arrive before the client deadline
    C-»S: Recover using the original operation identity
    S-»D: Read matching receipt under current access policy
    D--»S: Original application result
    S--»C: Result correlated to the new request
```

OBSERVATION	WHAT IT SUPPORTS	WHAT IT DOES NOT ESTABLISH
Client timeout	No answer received within the waiting limit	Whether the note committed
Handler entry log	Request reached that handler	Commit or successful tool execution
Matching committed receipt	Recorded outcome for that operation	Every external side effect succeeded
Missing receipt	No matching record visible to that lookup	No concurrent attempt can commit later
Matching note text	That text exists	Which intended operation created it

IDENTIFIER	QUESTION IT ANSWERS
JSON-RPC `id`	Which request does this response answer?
Resource handle	Which notebook does this call target?
Application operation key	Which intended append is being submitted again?

Preserve the original operation key and payload when following a documented retry contract. Check its retention window and current access rules. A payload conflict or expired operation key is not repaired by quietly generating a new key. A new key may describe a second action. Bound retry attempts and keep an unresolved outcome visible to the user.

This is a diagnostic reference, not a runnable lab dependency. Checked 2026-09-09 against HTTP retry semantics and JSON-RPC correlation.

Where this goes next. Lesson 2.4 makes a repeated business operation detectable with an atomic receipt, then fires duplicates, conflicting payloads and concurrent attempts at it in lab 2. The course: runsclean.com/courses/mcp-production/